AI Novel Scraper – Next Steps and Expansion Roadmap

This document outlines recommended next development phases for the AI Novel Scraper project. The goal is to progressively evolve the system from a simple crawler into a scalable, self■learning scraping platform capable of adapting to many web novel sites automatically.

Phase 1 – Stabilize the Core Scraper

- Add logging for crawler, extractor, and database actions.
- Add retry logic and error handling to avoid crashes.
- Normalize URLs and remove duplicate chapter entries.
- Improve chapter detection by scoring links based on text patterns (chapter, ch-, episode, etc.).
- Add domain-based rules so each site can store custom extraction selectors.

Phase 2 – Pattern Learning System

- Automatically store working CSS selectors per site.
- Track extraction success rate for each selector.
- If extraction fails, fall back to heuristic extraction.
- Persist patterns in a JSON or database table so they improve over time.

Phase 3 – Local AI Content Extraction

- Install a local LLM using Ollama (e.g., Llama3).
- Send HTML segments to the model asking it to extract chapter title and text.
- Convert the model response into structured JSON data.
- Use the AI extraction only when heuristic extraction fails.

Phase 4 – Self■Learning Site Adapter

- Allow users to provide one example chapter page.
- Analyze DOM structure to learn recurring patterns.

- Generate selectors for title, content, and next chapter links.
- Save the learned template for the entire site.

Phase 5 – Embedding Based Content Detection

- Use Sentence Transformers to create embeddings for page blocks.
- Identify which block is most similar to typical chapter text.
- Use cosine similarity to classify content blocks.

Phase 6 – Automatic Multi■Site Discovery

- Add crawler logic to explore links within novel domains.
- Detect new novels and automatically add them to the tracking list.
- Build a metadata index of novels, authors, and chapters.

Phase 7 – Notifications and User Interface

- Add Discord or Telegram notification support.
- Create a lightweight web dashboard using FastAPI.
- Allow users to track reading progress and mark chapters as read.

Phase 8 – Content Output Features

- Automatically generate EPUB files for each novel.
- Create a chapter archive folder system.
- Add optional audiobook generation using text■to■speech.

Phase 9 – Scaling the Scraper

- Introduce a queue system for scraping tasks.
- Run multiple workers for parallel scraping.
- Implement caching to avoid repeated downloads.

Phase 10 – Fully Autonomous Scraper

- Automatically adapt to layout changes.
- Relearn extraction patterns when scraping fails.
- Use AI classification to identify page structure types.
- Maintain a library of site templates.

Advanced Architecture Expansion

As the project matures, the system can evolve into a full scraping platform. The recommended architecture would include the following components:

- Crawler Cluster – multiple workers fetching pages.
- HTML Storage Layer – store raw pages for debugging.
- AI Extraction Engine – interprets page structure.
- Pattern Learning Engine – updates selectors automatically.
- Content Database – structured storage of novels and chapters.
- Notification Service – informs users when updates occur.
- Reader Interface – web application for reading chapters.

Following this roadmap gradually transforms the project from a simple scraper into a resilient, AI■assisted web■novel aggregation system. Each phase can be implemented independently, allowing development to progress step■by■step without breaking existing functionality.